

SQL JOINS

Complete Step-by-Step Guide

Tank54 Group | Oracle HR Schema

This guide teaches you SQL JOINS with a simple method. First, we show how to combine two tables by hand, step by step. Then, we show how each manual step becomes SQL code.

Topics: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, and Joining 3+ Tables

The Two Tables We Will Use

For all examples in this guide, we use two tables from the Oracle HR database: **EMPLOYEES** and **DEPARTMENTS**. The EMPLOYEES table has employee names and a department ID number. The DEPARTMENTS table has department names and an ID number. These two tables share one common column: **DEPARTMENT_ID**. This shared column is the bridge that connects the two tables together.

EMPLOYEES Table (5 rows):

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	JOB_ID	SALARY	DEPARTMENT_ID
100	Steven	King	AD_PRES	24000	90
101	Neena	Kochhar	AD_VP	17000	90
102	Lex	De Haan	AD_VP	17000	90
107	Diana	Lorentz	IT_PROG	4200	60
178	Kimberely	Grant	SA_REP	7000	null

DEPARTMENTS Table (4 rows):

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
60	IT	1400
90	Executive	1700
270	Payroll	1700

Key Idea: Both tables have a column called DEPARTMENT_ID. This is the shared column. We use this column to connect (join) the two tables. When the DEPARTMENT_ID in EMPLOYEES equals the DEPARTMENT_ID in DEPARTMENTS, we can put the employee name and department name on the same row.

How a JOIN Works - The General Idea

Before we write any SQL, let us think about how we would combine two tables by hand, on paper. This will help you understand what SQL does behind the scenes.

Step 1 - Find the shared column. First, look at both tables. Find a column that exists in both tables. In our case, both tables have DEPARTMENT_ID. This column is the "bridge" between the two tables. It is the intersection point.

Step 2 - Read a row from the first table. Take the first row from the EMPLOYEES table. For example, Steven King has DEPARTMENT_ID = 90.

Step 3 - Find the same value in the second table. Now look at the DEPARTMENTS table. Find the row where DEPARTMENT_ID = 90. We find: Executive. This means Steven works in the Executive department.

Step 4 - Write the result. Write down: Steven King | Executive. We put the department name next to the employee name. This is the first row of our result.

Step 5 - Repeat for all rows. Go back to the EMPLOYEES table and do the same for every row. Read the next employee, find their DEPARTMENT_ID in the DEPARTMENTS table, and write down the result. Do this until all employee rows are done.

Step 6 - The final result. After checking all rows, we have our result. But here is the important part: the final result depends on the JOIN type. Different JOIN types decide which rows appear in the result and which rows do not.

Important: Not every employee has a matching department. For example, Kimberly Grant has DEPARTMENT_ID = null. She has no department. And not every department has employees. Payroll (DEPARTMENT_ID = 270) has no employees. The JOIN type decides what happens with these unmatched rows.

1. INNER JOIN

How to Do It by Hand (No SQL)

INNER JOIN shows **only the rows where the shared column has a matching value in both tables**. If an employee has no department, that row will NOT appear. If a department has no employees, that row will NOT appear. Only rows that match in BOTH tables are shown.

Step 1: I look at both tables and find the shared column: DEPARTMENT_ID. This column exists in EMPLOYEES and also in DEPARTMENTS. This is my bridge.

Step 2: I read the first row of EMPLOYEES: Steven, DEPARTMENT_ID = 90. I go to DEPARTMENTS and look for DEPARTMENT_ID = 90. I find it: Executive. So I write: Steven | Executive.

Step 3: I read the next row: Neena, DEPARTMENT_ID = 90. Same department. I write: Neena | Executive.

Step 4: I read the next row: Lex, DEPARTMENT_ID = 90. I write: Lex | Executive.

Step 5: I read the next row: Diana, DEPARTMENT_ID = 60. I go to DEPARTMENTS and find DEPARTMENT_ID = 60: IT. I write: Diana | IT.

Step 6: I read the last row: Kimberly, DEPARTMENT_ID = null. I go to DEPARTMENTS and look for null. I do NOT find null in DEPARTMENTS. So I do NOT write anything for Kimberly. She is NOT in the result. This is because INNER JOIN only shows rows that match in BOTH tables.

Step 7: Payroll (DEPARTMENT_ID = 270) has no employees. No employee row has DEPARTMENT_ID = 270. So Payroll is NOT in the result either.

Final Result (INNER JOIN):

First Name	Last Name	Department Name
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
Diana	Lorentz	IT

Notice: Only 4 rows in the result. Kimberly is missing (no department). Payroll is missing (no employees). INNER JOIN = only matched rows from both tables.

Now Let Us Write the SQL

Now we translate our manual steps into SQL code. Look at each part of the SQL and see which manual step it comes from.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
      ON e.department_id = d.department_id
ORDER BY e.last_name;
```

How SQL Maps to Our Manual Steps:

- **FROM employees e** = "Start with the EMPLOYEES table" (Step 2). "e" is a short name for employees.
- **INNER JOIN departments d** = "Go to the DEPARTMENTS table to find the match" (Step 2-3). "d" is a short name for departments.
- **ON e.department_id = d.department_id** = "Match the shared column values" (Step 2-3). This is our bridge. "Find the same DEPARTMENT_ID in both tables."
- **INNER JOIN** = "Only show rows where the match exists in BOTH tables" (Step 6-7). No match = no row in the result.
- **SELECT e.first_name, e.last_name, d.department_name** = "Write down these columns" (Step 4). "e." means the column comes from employees. "d." means it comes from departments.
- **ORDER BY e.last_name** = "Sort the result by last name".

Sample Task (with Explanation)

Task: Write a query to show the employee first name, job id, and department name. Show only employees who have a department.

How to solve it:

I need data from two tables: EMPLOYEES (for first_name and job_id) and DEPARTMENTS (for department_name). The shared column is DEPARTMENT_ID. I want only matched rows, so I use INNER JOIN. I write "e" for employees and "d" for departments to keep the query short.

Answer:

```
SELECT e.first_name,  
       e.job_id,  
       d.department_name  
FROM employees e  
INNER JOIN departments d  
      ON e.department_id = d.department_id;
```

Practice Tasks (try these yourself)

Task 1: Show the employee last name and department name. Use INNER JOIN.

Task 2: Show the employee first name, salary, and department name for employees who have a department.

Task 3: Show the department name and location id for departments that have at least one employee. Use INNER JOIN.

Task 4: Show the employee first name, last name, and department name. Sort the result by first name.

Task 5: Show the job id and department name for all employees who have a department. Use INNER JOIN.

2. LEFT JOIN

How to Do It by Hand (No SQL)

LEFT JOIN shows **ALL rows from the first table (left table)**. If a row in the first table has a match in the second table, we show the match. If a row in the first table does NOT have a match, we show null for the second table columns. In our case, the left table is EMPLOYEES (the table after FROM). The right table is DEPARTMENTS (the table after JOIN).

Step 1: I find the shared column: DEPARTMENT_ID. Same as before.

Step 2: I read the first row of EMPLOYEES: Steven, DEPARTMENT_ID = 90. I find DEPARTMENT_ID = 90 in DEPARTMENTS: Executive. I write: Steven | Executive.

Step 3: Next row: Neena, DEPARTMENT_ID = 90. I write: Neena | Executive.

Step 4: Next row: Lex, DEPARTMENT_ID = 90. I write: Lex | Executive.

Step 5: Next row: Diana, DEPARTMENT_ID = 60. I find IT in DEPARTMENTS. I write: Diana | IT.

Step 6: Next row: Kimberly, DEPARTMENT_ID = null. I look in DEPARTMENTS for null. I do NOT find it. But this time, I STILL write Kimberly in my result, with null for department name. Why? Because LEFT JOIN keeps ALL rows from the left table (EMPLOYEES), even if there is no match. So I write: Kimberly | null.

Step 7: Payroll (DEPARTMENT_ID = 270) has no employees. Since Payroll is in the right table and has no match in the left table, it does NOT appear in the result. LEFT JOIN keeps all rows from the LEFT table only.

Final Result (LEFT JOIN):

First Name	Last Name	Department Name
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
Diana	Lorentz	IT
Kimberly	Grant	null

Notice: 5 rows now! Kimberly appears with null department. Payroll is still missing (right table, no match). LEFT JOIN = all rows from left table + matched rows from right table.

Now Let Us Write the SQL

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
LEFT JOIN departments d
      ON e.department_id = d.department_id
ORDER BY e.last_name;
```

How SQL Maps to Our Manual Steps:

- FROM employees e = "Start with EMPLOYEES table" (left table).
- LEFT JOIN departments d = "Go to DEPARTMENTS" (right table). LEFT JOIN means: keep ALL rows from the left table.
- ON e.department_id = d.department_id = "Match the shared column" (the bridge).
- If a match is found: show the department name (Steps 2-5).
- If NO match is found: show null for department (Step 6, Kimberly).
- The right table rows with no match (Payroll) are NOT shown.

Sample Task (with Explanation)

Task: Show ALL employees with their department name. Even if an employee has no department, show the employee with null for department.

How to solve it:

I want ALL employees, even those without a department. So I start from EMPLOYEES (left table) and use LEFT JOIN. This keeps all employees. If an employee has no matching department, the department_name will be null.

Answer:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
LEFT JOIN departments d  
      ON e.department_id = d.department_id;
```

Practice Tasks (try these yourself)

Task 1: Show all employees and their department name. Use LEFT JOIN.

Task 2: Show the employee first name, salary, and department name for ALL employees.

Task 3: Show the employee last name, job id, and department name for all employees, even those with no department.

Task 4: Show the employee first name and department location id for all employees.

Task 5: Write a LEFT JOIN query to show department name and employee last name for all employees.

3. RIGHT JOIN

How to Do It by Hand (No SQL)

RIGHT JOIN shows **ALL rows from the second table (right table)**. If a row in the right table has a match in the left table, we show the match. If a row in the right table does NOT have a match, we show null for the left table columns. The left table is EMPLOYEES (after FROM). The right table is DEPARTMENTS (after JOIN). RIGHT JOIN is the opposite of LEFT JOIN.

Step 1: I find the shared column: DEPARTMENT_ID.

Step 2: I read the first row of DEPARTMENTS: Administration, DEPARTMENT_ID = 10. I look in EMPLOYEES for DEPARTMENT_ID = 10. I do NOT find it. But this time, I still write Administration in my result with null for employee name. RIGHT JOIN keeps ALL rows from the right table (DEPARTMENTS). So I write: null | Administration.

Step 3: Next row: IT, DEPARTMENT_ID = 60. I find Diana in EMPLOYEES with DEPARTMENT_ID = 60. I write: Diana | IT.

Step 4: Next row: Executive, DEPARTMENT_ID = 90. I find Steven, Neena, and Lex with DEPARTMENT_ID = 90. I write 3 rows: Steven | Executive, Neena | Executive, Lex | Executive.

Step 5: Last row: Payroll, DEPARTMENT_ID = 270. I look in EMPLOYEES for DEPARTMENT_ID = 270. I do NOT find it. I still write: null | Payroll. RIGHT JOIN keeps all departments.

Step 6: Kimberly has DEPARTMENT_ID = null. She is in the left table. Since no department has null as DEPARTMENT_ID, she does NOT appear. RIGHT JOIN keeps all rows from the RIGHT table, not the left.

Final Result (RIGHT JOIN):

First Name	Last Name	Department Name
null	null	Administration
Diana	Lorentz	IT
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
null	null	Payroll

Notice: 6 rows. Administration and Payroll appear with null employee names. Kimberly is missing (left table, no match). RIGHT JOIN = all rows from right table + matched rows from left table.

Now Let Us Write the SQL

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
RIGHT JOIN departments d
      ON e.department_id = d.department_id
ORDER BY d.department_name;
```

How SQL Maps to Our Manual Steps:

- FROM employees e = "EMPLOYEES is the left table".
- RIGHT JOIN departments d = "DEPARTMENTS is the right table. Keep ALL its rows."
- ON e.department_id = d.department_id = "Match the shared column".
- If a department has a matching employee: show both (Steps 3-4).
- If a department has NO matching employee: show null for employee (Steps 2, 5).
- Employees with no matching department are NOT shown (Step 6, Kimberly).

Sample Task (with Explanation)

Task: Show ALL departments with their employee names. If a department has no employees, show the department with null for employee.

How to solve it:

I want ALL departments, even those without employees. I use RIGHT JOIN because DEPARTMENTS is the second table (right table). RIGHT JOIN keeps all rows from the right table.

Answer:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
RIGHT JOIN departments d  
      ON e.department_id = d.department_id;
```

Practice Tasks (try these yourself)

Task 1: Show all departments and the employees who work there. Use RIGHT JOIN.

Task 2: Show the department name and employee salary for ALL departments.

Task 3: Show the department location id and employee first name for all departments.

Task 4: Show the department name and employee job id. Include departments with no employees.

Task 5: Write a RIGHT JOIN query to show employee last name and department name for all departments.

4. FULL OUTER JOIN

How to Do It by Hand (No SQL)

FULL OUTER JOIN shows **ALL rows from both tables**. If a row matches, we show the match. If a row in the left table has no match, we show it with null for the right columns. If a row in the right table has no match, we show it with null for the left columns. Nothing is left out. Everything appears in the result.

Step 1: I find the shared column: DEPARTMENT_ID.

Step 2: I read all employee rows and find their departments, just like INNER JOIN. Steven - Executive, Neena - Executive, Lex - Executive, Diana - IT.

Step 3: I also check for employees with NO department match. Kimberly has DEPARTMENT_ID = null. No department has null. But FULL OUTER JOIN keeps her. I write: Kimberly | null.

Step 4: I check for departments with NO employee match. Administration (DEPARTMENT_ID = 10) has no employees. I write: null | Administration. Payroll (DEPARTMENT_ID = 270) has no employees. I write: null | Payroll.

Step 5: The final result has everything. All matched rows plus all unmatched rows from both tables. Nothing is missing.

Final Result (FULL OUTER JOIN):

First Name	Last Name	Department Name
null	null	Administration
Diana	Lorentz	IT
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
null	null	Payroll
Kimberely	Grant	null

Notice: 7 rows. Everything is here! Matched rows (4) + unmatched from left (Kimberely) + unmatched from right (Administration, Payroll). FULL OUTER JOIN = INNER JOIN + LEFT JOIN extra rows + RIGHT JOIN extra rows.

Now Let Us Write the SQL

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
FULL OUTER JOIN departments d
  ON e.department_id = d.department_id
ORDER BY d.department_name, e.last_name;
```

How SQL Maps to Our Manual Steps:

- **FROM employees e** = "Start with EMPLOYEES".
- **FULL OUTER JOIN departments d** = "Keep ALL rows from BOTH tables".
- **ON e.department_id = d.department_id** = "Match the shared column".
- Matched rows: show both names (Step 2).
- Unmatched employees: show employee with null department (Step 3).
- Unmatched departments: show department with null employee (Step 4).

Sample Task (with Explanation)

Task: Show ALL employees and ALL departments. Show employee name and department name together. Include employees with no department and departments with no employees.

How to solve it:

I want everything from both tables. Nothing should be missing. So I use FULL OUTER JOIN. This keeps all rows from both tables. Matched rows show both names. Unmatched rows show null for the missing side.

Answer:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
FULL OUTER JOIN departments d  
  ON e.department_id = d.department_id;
```

Practice Tasks (try these yourself)

Task 1: Show all employees and all departments. Use FULL OUTER JOIN.

Task 2: Show the employee first name, salary, and department name for ALL employees and ALL departments.

Task 3: Show department name and employee job id. Include all rows from both tables.

Task 4: Show all employee last names and all department names using FULL OUTER JOIN.

Task 5: Write a FULL OUTER JOIN to show department location id and employee first name for everything.

5. Joining More Than 2 Tables

Sometimes we need data from 3 or more tables. For example: an employee works in a department, and the department is in a city. If we want to see the employee name, department name, AND city, we need 3 tables: EMPLOYEES, DEPARTMENTS, and LOCATIONS. We do the same thing as before, but we add more steps.

The 3 Tables and How They Connect:

The tables connect like a chain. Each table links to the next one by a shared column:

LOCATIONS Table (sample rows):

LOCATION_ID	CITY	COUNTRY_ID
1400	South San Francisco	US
1700	Seattle	US

The Chain:

EMPLOYEES (DEPARTMENT_ID) --> DEPARTMENTS (DEPARTMENT_ID, LOCATION_ID) --> LOCATIONS (LOCATION_ID)

Table 1 connects to Table 2 by DEPARTMENT_ID. Table 2 connects to Table 3 by LOCATION_ID.

How to Do It by Hand (No SQL)

Imagine 3 papers on your desk. Paper 1 = EMPLOYEES. Paper 2 = DEPARTMENTS. Paper 3 = LOCATIONS. You want one list: employee name, department name, city.

Step 1: I find the shared columns. EMPLOYEES and DEPARTMENTS share DEPARTMENT_ID. DEPARTMENTS and LOCATIONS share LOCATION_ID. This is my chain of connections.

Step 2: I read the first row of EMPLOYEES: Steven, DEPARTMENT_ID = 90. I go to DEPARTMENTS and find DEPARTMENT_ID = 90: Executive, LOCATION_ID = 1700.

Step 3: Now I use LOCATION_ID = 1700 and go to LOCATIONS. I find LOCATION_ID = 1700: Seattle.

Step 4: I write the result: Steven | Executive | Seattle. This is the first row. I followed the chain: EMPLOYEES --> DEPARTMENTS --> LOCATIONS.

Step 5: I go back to EMPLOYEES and read the next row: Neena, DEPARTMENT_ID = 90. I repeat steps 2-4: find Executive, LOCATION_ID = 1700, find Seattle. I write: Neena | Executive | Seattle.

Step 6: I continue for all employee rows. For Diana (DEPARTMENT_ID = 60): IT, LOCATION_ID = 1400, South San Francisco. I write: Diana | IT | South San Francisco.

Step 7: For Kimberly (DEPARTMENT_ID = null): I cannot find a department. So I write: Kimberly | null | null. She has no department and no city.

Final Result:

First Name	Last Name	Department	City
Steven	King	Executive	Seattle
Neena	Kochhar	Executive	Seattle
Lex	De Haan	Executive	Seattle
Diana	Lorentz	IT	South San Francisco
Kimberely	Grant	null	null

Notice: Data from 3 tables in one result! We followed the chain for each employee: EMPLOYEES --> DEPARTMENTS --> LOCATIONS. Kimberly shows null for both department and city because she has no DEPARTMENT_ID.

Now Let Us Write the SQL

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name,
       l.city
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
LEFT JOIN locations l
  ON d.location_id = l.location_id
```



```
ORDER BY e.last_name;
```

How SQL Maps to Our Manual Steps:

- **FROM employees e** = "Start with EMPLOYEES" (Step 2).
- **LEFT JOIN departments d ON e.department_id = d.department_id** = "Go to DEPARTMENTS, find the match using DEPARTMENT_ID" (Step 2). We use LEFT JOIN to keep Kimberly.
- **LEFT JOIN locations l ON d.location_id = l.location_id** = "Go to LOCATIONS, find the match using LOCATION_ID" (Step 3). We use d.location_id (from the PREVIOUS table, DEPARTMENTS) to connect to LOCATIONS.
- **SELECT ... e.first_name, e.last_name, d.department_name, l.city** = "Write down these 4 columns" (Step 4).

Rule: When you join N tables, you need N-1 JOIN lines. 3 tables = 2 JOINS. 4 tables = 3 JOINS. Each JOIN uses a shared column.

Sample Task (with Explanation)

Task: Show the employee first name, department name, and city. Show ALL employees, even those with no department.

How to solve it:

I need 3 tables: EMPLOYEES, DEPARTMENTS, LOCATIONS. I want ALL employees, so I use LEFT JOIN for the first JOIN (employees to departments). For the second JOIN (departments to locations), I also use LEFT JOIN so that if a department has no location, the city will be null. The chain is: EMPLOYEES --DEPARTMENT_ID--> DEPARTMENTS --LOCATION_ID--> LOCATIONS.

Answer:

```
SELECT e.first_name,
       d.department_name,
       l.city
FROM employees e
LEFT JOIN departments d
      ON e.department_id = d.department_id
LEFT JOIN locations l
      ON d.location_id = l.location_id;
```

Practice Tasks (try these yourself)

Task 1: Show employee last name, department name, and city. Use 3 tables with LEFT JOIN.

Task 2: Show employee first name, salary, department name, and city for ALL employees.

Task 3: Show employee job id, department name, and city. Include employees with no department.

Task 4: Show the department name and city for departments that have at least one employee. Use INNER JOIN for 3 tables.

Task 5: Write a query with 3 tables to show employee first name, department name, and country id. Use LEFT JOIN.

Understanding Chained LEFT JOINS

A visual explanation of how each JOIN builds on the previous result

The Common Question:

When we write: `LEFT JOIN hr.locations l ON d.location_id = l.location_id`

Does `d.location_id` come from the original `hr.departments` table?

Or does it come from the result of the previous JOIN?

The Answer: It comes from the RESULT of the previous JOIN!

SQL processes JOINS one at a time, from top to bottom. Each JOIN takes the cumulative result of ALL previous JOINS as its left table.

Step 1: `FROM employees e`

Start with the employees table as the base.

Step 2: `LEFT JOIN departments d ON e.department_id = d.department_id`

Left table = employees. Result 1 = employees + departments.

Now `d.location_id` is available in Result 1.

Step 3: `LEFT JOIN jobs j ON e.job_id = j.job_id`

Left table = Result 1 (employees + departments).

`e.job_id` and `d.location_id` are BOTH from the cumulative result.

Step 4: `LEFT JOIN locations l ON d.location_id = l.location_id`

Left table = Result 2. `d.location_id` comes from Result 2.

It does NOT read from the original departments table!

Step 5: `LEFT JOIN countries c ON l.country_id = c.country_id`

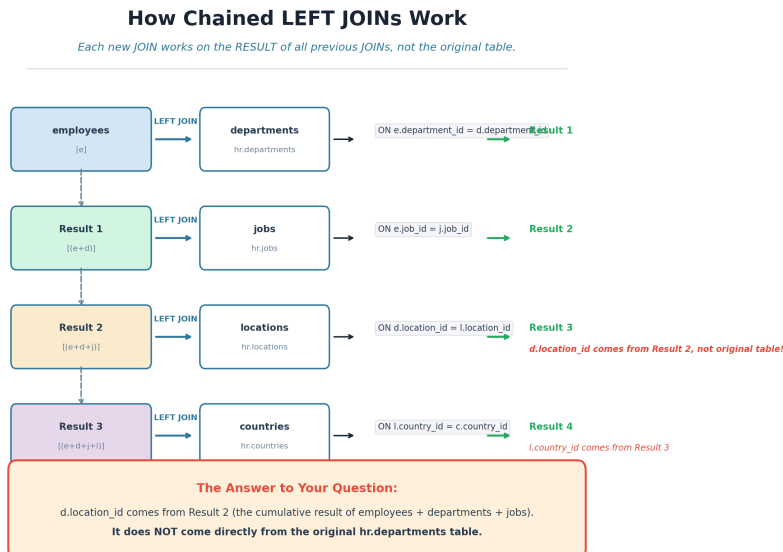
Left table = Result 3. `l.country_id` comes from Result 3.

Step 6: `LEFT JOIN regions r ON r.region_id = c.region_id`

Left table = Result 4. `c.region_id` comes from Result 4.

KEY RULE: Every column reference in a chained LEFT JOIN refers to the CUMULATIVE RESULT, NOT the original table.

Visual: How Each JOIN Builds on the Previous Result



The Full Query:

```
SELECT first_name, last_name, salary, job_title,  
       department_name, city, country_name, region_name  
FROM hr.employees e  
LEFT JOIN hr.departments d ON e.department_id = d.department_id  
LEFT JOIN hr.jobs j ON e.job_id = j.job_id  
LEFT JOIN hr.locations l ON d.location_id = l.location_id  
LEFT JOIN hr.countries c ON l.country_id = c.country_id  
LEFT JOIN hr.regions r ON r.region_id = c.region_id  
WHERE department_name IS NOT NULL;
```

Quick Summary

JOIN Type	Kimberely (no dept)	Payroll (no employees)	Matched rows (e.g. Steven)
INNER JOIN	NO	NO	YES
LEFT JOIN	YES	NO	YES
RIGHT JOIN	NO	YES	YES
FULL OUTER JOIN	YES	YES	YES

When to Use Which JOIN?

Situation	Use This JOIN
Only employees who have a department.	INNER JOIN
ALL employees, even with no department.	LEFT JOIN
ALL departments, even with no employees.	RIGHT JOIN
Everything from both tables.	FULL OUTER JOIN

Multiple Table JOINS

When you join N tables, you need N-1 JOIN lines. Each JOIN needs an ON condition that connects to the previous table in the chain.

Tables	JOIN Lines	Example Chain
2	1	employees --> departments
3	2	employees --> departments --> locations
4	3	employees --> departments --> locations --> countries

Remember: Each new JOIN uses a shared column from the PREVIOUS table to connect to the NEW table. This forms a chain. SQL follows the chain from left to right.